Міністерство освіти і науки України

Львівський національний університет імені Івана Франка

Факультет електроніки та комп’ютерних технологій

**ЗВІТ**

про виконання лабораторної роботи №9

**“ Паралельність, багатопроцесність та оптимізація ”**

**Виконала:**

Студентка групи ФЕП-33с

Най Анастасія

**Перевірив:**

Проф. Свелеба С.А.

**Львів-2026**

**Мета роботи**: ознайомитися з основними підходами до паралельного та конкурентного виконання програм у Python, дослідити вплив Global Interpreter Lock (GIL) на виконання CPU-bound та I/O-bound задач, навчитися використовувати threading, multiprocessing, ThreadPoolExecutor і ProcessPoolExecutor.

**Варіант 10. Система обліку складських запасів**

Предметна область Lab_01: товар має код, назву, кількість і ціну.

Зберегти базові можливості: надходження; списання; пошук; загальна вартість складу; визначення низьких залишків.

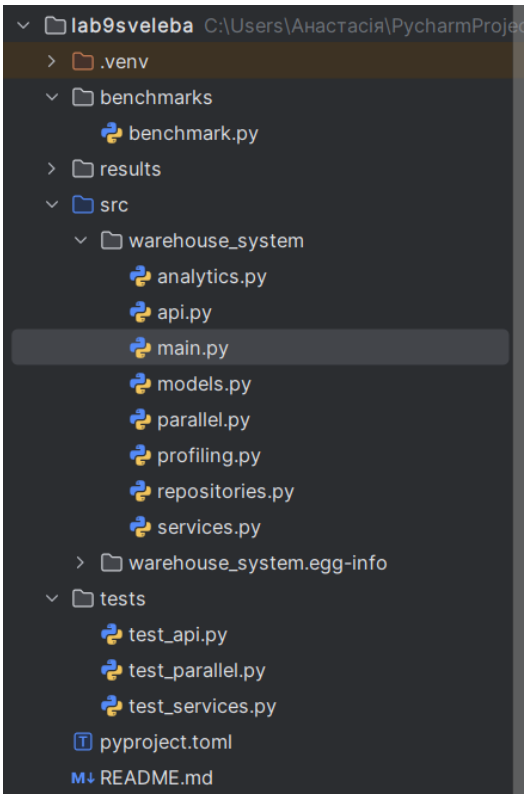
На великому складському каталозі профілювати загальну вартість, low-stock search, статистику руху та обробку великого журналу надходжень/списань.

ThreadPoolExecutor використати для паралельного читання декількох журналів; ProcessPoolExecutor і NumPy - для CPU-bound агрегації та числової статистики.

Продемонструвати race condition при одночасній зміні in-memory залишку і виправити її через Lock. Caching застосувати до складської статистики з invalidation після руху товару.

**Хід роботи:**

1. Спочатку я створила робочу папку проєкту. Організувала професійну структуру за стандартом src-layout: у папці src/warehouse_system/ зосередила весь програмний код, залишивши корінь проєкту для конфігураційних файлів (pyproject.toml, README.md) та каталогу тестів tests/ , та каталог benchmarks/



2. У файлі pyproject.toml я налаштувала метадані пакета, вказала версію проєкту, інформацію про автора, мінімальну версію Python, а також основні залежності (FastAPI, Uvicorn, Pydantic, NumPy, httpx). Також додала необов'язкові розробницькі залежності (pytest, pytest-asyncio, mypy) та вказала конфігурацію збірки проєкту за допомогою setuptools через src-layout.

```
[build-system]
requires = ["setuptools>=61.0"]
build-backend = "setuptools.build_meta"

[project]
name = "warehouse_system"
version = "0.1.0"
description = "Система обліку складських запасів з підтримкою паралельних
обчислень та профілювання"
authors = [{ name = "Анастасія Най" }]
requires-python = ">=3.10"
dependencies = [
    "fastapi>=0.100.0",
    "uvicorn>=0.22.0",
    "pydantic>=2.0.0",
    "numpy>=1.24.0",
    "httpx>=0.24.0"
]

[project.optional-dependencies]
dev = [
    "pytest>=7.0.0",
    "pytest-asyncio>=0.21.0",
    "mypy>=1.0.0"
]

[tool.setuptools.packages.find]
where = ["src"]
```

3. У файлі README.md я оформила документацію до проєкту: додала короткий опис системи обліку складських запасів, інструкцію зі розгортання та створення віртуального середовища .venv, команди для встановлення залежностей у режимі редагування (pip install -e .), а також інструкції для запуску API-сервера, тестів, бенчмарків та профілювання.

````
# Warehouse System (Варіант 10: Система обліку складських запасів)

Модульна система обліку складських запасів із підтримкою багатопотоковості,
багатопроцесності, векторизації через NumPy, профілювання часу/пам'яті та
кешування з інвалідацією.

## Встановлення та запуск

```bash
# Створення та активація віртуального середовища
python -m venv .venv
source .venv/bin/activate  # для Linux/macOS
# .venv\Scripts\activate   # для Windows

# Встановлення залежностей
pip install -e ".[dev]"

# Запуск API сервера
uvicorn warehouse_system.main:app --reload
````
````

```
# Запуск тестів
pytest

# Запуск бенчмарків
python benchmarks/benchmark.py

# Запуск профілювання
python -m warehouse_system.profiling
```

4. У файлі models.py я реалізувала Pydantic-моделі для валідації, парсингу та типізації складських даних. Створила базову модель WarehouseItemBase, моделі для створення (WarehouseItemCreate) та часткового оновлення товарів (WarehouseItemUpdate з optional-полями для PATCH-запитів), підсумкову модель WarehouseItem із конфігурацією model_config = {"from_attributes": True}, а також модель StockMovement для фіксації надходження й списання товарів.

```
from __future__ import annotations
from typing import Optional
from pydantic import BaseModel, Field


class WarehouseItemBase(BaseModel):
    code: str = Field(..., description="Унікальний код товару")
    name: str = Field(..., description="Назва товару")
    quantity: int = Field(..., ge=0, description="Кількість на складі")
    price: float = Field(..., gt=0, description="Ціна за одиницю")
    category: str = Field(default="General", description="Категорія товару")


class WarehouseItemCreate(WarehouseItemBase):
    pass


class WarehouseItemUpdate(BaseModel):
    name: Optional[str] = None
    quantity: Optional[int] = Field(None, ge=0)
    price: Optional[float] = Field(None, gt=0)
    category: Optional[str] = None


class WarehouseItem(WarehouseItemBase):
    model_config = {"from_attributes": True}


class StockMovement(BaseModel):
    code: str
    change: int  # Додатне значення — надходження, від'ємне — списання
```

5. У файлі repositories.py я реалізувала in-memory репозиторій WarehouseRepository для збереження товарів у словнику. Для забезпечення безпеки роботи у багатопотоковому середовищі я використала примітив синхронізації threading.Lock, який захищає CRUD-операції (додавання, отримання, оновлення, видалення та очищення) від race condition при одночасному доступі.

```
from __future__ import annotations
from typing import Optional
```

```python
from threading import Lock
from warehouse_system.models import WarehouseItem, WarehouseItemCreate,
WarehouseItemUpdate


class WarehouseRepository:
    """In-memory репозиторій складських товарів із потокобезпечністю."""

    def __init__(self) -> None:
        self._items: dict[str, WarehouseItem] = {}
        self._lock = Lock()

    def add(self, item_data: WarehouseItemCreate) -> WarehouseItem:
        with self._lock:
            if item_data.code in self._items:
                raise ValueError(f"Товар з кодом {item_data.code} вже існує.")
            item = WarehouseItem(**item_data.model_dump())
            self._items[item.code] = item
            return item

    def get_by_code(self, code: str) -> Optional[WarehouseItem]:
        with self._lock:
            return self._items.get(code)

    def get_all(self) -> list[WarehouseItem]:
        with self._lock:
            return list(self._items.values())

    def update(self, code: str, update_data: WarehouseItemUpdate) ->
Optional[WarehouseItem]:
        with self._lock:
            item = self._items.get(code)
            if not item:
                return None

            data = item.model_dump()
            changes = update_data.model_dump(exclude_unset=True)
            data.update(changes)

            updated_item = WarehouseItem(**data)
            self._items[code] = updated_item
            return updated_item

    def delete(self, code: str) -> bool:
        with self._lock:
            if code in self._items:
                del self. items[code]
                return True
            return False

    def clear(self) -> None:
        with self._lock:
            self._items.clear()
```

6. У файлі analytics.py я реалізувала аналітичні алгоритми та генерацію даних. Написала функцію generate_large_dataset для створення масивів товарів (від 10 000 до 1 000 000+), baseline-функцію calculate_total_value_python на циклах Python, оптимізовану функцію calculate_total_value_numpy з векторизованими обчисленнями через NumPy, а також CPU-bound функцію cpu_bound_heavy_item_processing з математичними обчисленнями для профілювання завантаження CPU. Додатково реалізувала механізм кешування складської статистики через

@lru_cache із версіюванням та інвалідацією кешу (invalidate_warehouse_cache).

```
from __future__ import annotations
import random
import math
from functools import lru_cache
import numpy as np


def generate_large_dataset(count: int, seed: int = 42) -> list[dict]:
    """Генерація великого тестового масиву складських товарів."""
    random.seed(seed)
    categories = ["Electronics", "Groceries", "Hardware", "Clothing",
"Stationery"]
    return [
        {
            "code": f"PRD-{i:07d}",
            "name": f"Product_{i}",
            "quantity": random.randint(0, 500),
            "price": round(random.uniform(10.0, 5000.0), 2),
            "category": random.choice(categories)
        }
        for i in range(count)
    ]


def calculate_total_value_python(items: list[dict]) -> float:
    """Baseline: розрахунок загальної вартості складу через цикл Python."""
    total = 0.0
    for item in items:
        total += item["quantity"] * item["price"]
    return total


def calculate_total_value_numpy(items: list[dict]) -> float:
    """NumPy Оптимізація: векторизований розрахунок загальної вартості
складу."""
    quantities = np.fromiter((item["quantity"] for item in items),
dtype=np.float64, count=len(items))
    prices = np.fromiter((item["price"] for item in items), dtype=np.float64,
count=len(items))
    return float(np.dot(quantities, prices))


def find_low_stock_python(items: list[dict], threshold: int = 10) -> list[dict]:
    """Baseline: пошук товарів з низьким залишком."""
    return [item for item in items if item["quantity"] <= threshold]


def cpu_bound_heavy_item_processing(item_tuple: tuple[int, float]) -> float:
    """Складна CPU-bound обробка одного товару для вимірювання задіяності
ядер."""
    qty, price = item_tuple
    val = float(qty * price)
    for _ in range(100):
        val = math.sin(val) ** 2 + math.cos(val) ** 2 + math.sqrt(abs(val) +
1.0)
    return val


# Система кешування з версіюванням для інвалідації
_cache_version = 0


def invalidate_warehouse_cache() -> None:
```

```
    """Інвалідація (очищення) кешу складської статистики."""
    global _cache_version
    _cache_version += 1
    get_cached_warehouse_stats.cache_clear()


@lru_cache(maxsize=128)
def get_cached_warehouse_stats(version: int, total_items: int, total_value:
float) -> dict:
    """Кешована статистика складу."""
    return {
        "status": "computed",
        "cache_version": version,
        "total_items": total_items,
        "total_value": round(total_value, 2)
    }
```

7. У файлі parallel.py я реалізувала конкурентні та паралельні обчислення. Написала функцію process_items_threads для запуску обробки даних через ThreadPoolExecutor, функцію process_items_processes із використанням ProcessPoolExecutor та параметра chunksize для розділення задач між декількома процесами в обхід GIL. Створила приклад паралельного читання журналів файлів (read_logs_parallel) як I/O-bound експеримент, а також наочно продемонструвала наявність race condition (unsafe_increment_stock) та її виправлення за допомогою threading.Lock (safe_increment_stock).

```
from __future__ import annotations
from concurrent.futures import ThreadPoolExecutor, ProcessPoolExecutor
from threading import Lock
from pathlib import Path
from warehouse_system.analytics import cpu_bound_heavy_item_processing


def process_items_threads(items: list[dict], workers: int = 4) -> list[float]:
    """Паралельна обробка товарів через ThreadPoolExecutor."""
    data = [(item["quantity"], item["price"]) for item in items]
    with ThreadPoolExecutor(max_workers=workers) as executor:
        return list(executor.map(cpu_bound_heavy_item_processing, data))


def process_items_processes(items: list[dict], workers: int = 4) -> list[float]:
    """Багатопроцесна обробка CPU-bound задач через ProcessPoolExecutor."""
    data = [(item["quantity"], item["price"]) for item in items]
    with ProcessPoolExecutor(max_workers=workers) as executor:
        return list(executor.map(cpu_bound_heavy_item_processing, data,
chunksize=100))


def read_log_file(path: Path) -> str:
    """I/O-bound функція читання одного файлу журналу."""
    return path.read_text(encoding="utf-8")


def read_logs_parallel(paths: list[Path], workers: int = 4) -> list[str]:
    """Паралельне читання багатьох журналів через ThreadPoolExecutor."""
    with ThreadPoolExecutor(max_workers=workers) as executor:
        return list(executor.map(read_log_file, paths))


# Демонстрація Race Condition та її усунення через Lock
```

```
stock_counter = 0
counter_lock = Lock()


def unsafe_increment_stock() -> None:
    """Непотокобезпечна зміна стану (викликає Race Condition)."""
    global stock_counter
    current = stock_counter
    # Моделювання перемикання контексту
    stock_counter = current + 1


def safe_increment_stock() -> None:
    """Потокобезпечна зміна стану через Lock."""
    global stock_counter
    with counter_lock:
        current = stock_counter
        stock_counter = current + 1
```

8. У файлі services.py я реалізувала бізнес-логіку системи у класі WarehouseService. Додала методи для створення товарів, отримання списку, внесення змін, проведення надходжень/списань та отримання статистики складу. Забезпечила автоматичний виклик інвалідації кешу при будь-якій зміні складських залишків чи характеристик товарів.

```
from __future__ import annotations
from typing import Optional
from warehouse_system.repositories import WarehouseRepository
from warehouse_system.models import WarehouseItem, WarehouseItemCreate,
WarehouseItemUpdate
from warehouse_system.analytics import (
    calculate_total_value_numpy,
    get_cached_warehouse_stats,
    invalidate_warehouse_cache,
    _cache_version
)


class WarehouseService:
    def __init__(self, repository: WarehouseRepository) -> None:
        self.repo = repository

    def add_item(self, item_data: WarehouseItemCreate) -> WarehouseItem:
        item = self.repo.add(item_data)
        invalidate_warehouse_cache()
        return item

    def get_item(self, code: str) -> Optional[WarehouseItem]:
        return self.repo.get_by_code(code)

    def list_items(self) -> list[WarehouseItem]:
        return self.repo.get_all()

    def update_item(self, code: str, update_data: WarehouseItemUpdate) ->
Optional[WarehouseItem]:
        item = self.repo.update(code, update_data)
        if item:
            invalidate_warehouse_cache()
        return item

    def record_movement(self, code: str, change: int) ->
Optional[WarehouseItem]:
        item = self.repo.get_by_code(code)
```

```
        if not item:
            return None
        new_qty = item.quantity + change
        if new_qty < 0:
            raise ValueError("Кількість товару не може бути від'ємною.")

        updated = self.repo.update(code, WarehouseItemUpdate(quantity=new_qty))
        invalidate_warehouse_cache()
        return updated

    def get_stats(self) -> dict:
        items = [item.model_dump() for item in self.repo.get_all()]
        total_value = calculate_total_value_numpy(items) if items else 0.0
        return get_cached_warehouse_stats(_cache_version, len(items),
total_value)
```

9. У файлі profiling.py я налаштувала інструменти для виявлення bottlenecks у програмі. Написала функцію profile_execution_time, яка використовує cProfile та pstats для аналізу накопиченого часу виконання функцій (cumtime), а також функцію profile_memory_usage, яка за допомогою tracemalloc вимірює поточне та пікове використання пам'яті Python-процесом.

```
import cProfile
import pstats
import tracemalloc
from warehouse_system.analytics import (
    generate_large_dataset,
    calculate_total_value_python,
)


def profile_execution_time() -> None:
    """Аналіз продуктивності та виявлення bottlenecks за допомогою cProfile."""
    print("=== cProfile: Аналіз часу виконання ===")
    dataset = generate_large_dataset(100_000)

    profiler = cProfile.Profile()
    profiler.enable()

    _ = calculate_total_value_python(dataset)

    profiler.disable()
    stats = pstats.Stats(profiler)
    stats.sort_stats("cumulative")
    stats.print_stats(10)


def profile_memory_usage() -> None:
    """Аналіз використання пам'яті за допомогою tracemalloc."""
    print("\n=== tracemalloc: Аналіз використання пам'яті ===")
    tracemalloc.start()

    dataset = generate_large_dataset(200_000)
    current, peak = tracemalloc.get_traced_memory()

    print(f"Поточна виділена пам'ять: {current / (1024 * 1024):.2f} MB")
    print(f"Пікова виділена пам'ять:    {peak / (1024 * 1024):.2f} MB")

    tracemalloc.stop()


if __name__ == "__main__":
```

```
    profile_execution_time()
    profile_memory_usage()
```

10. У файлі api.py я створила REST API на базі FastAPI. Описала ендпоінти для створення товару (POST /items із кодом 201 Created), отримання списку та одного товару за кодом (GET /items, GET /items/{code}), часткового оновлення (PATCH /items/{code}), проведення руху товарів (POST /items/movement) та отримання кешованої аналітики (GET /analytics/stats), додавши відповідні обробники HTTP-помилок.

```
from fastapi import FastAPI, HTTPException, status
from warehouse_system.repositories import WarehouseRepository
from warehouse_system.services import WarehouseService
from warehouse_system.models import (
    WarehouseItem,
    WarehouseItemCreate,
    WarehouseItemUpdate,
    StockMovement
)

app = FastAPI(title="Warehouse Management System API", version="0.1.0")

repo = WarehouseRepository()
service = WarehouseService(repo)


@app.post("/items", response_model=WarehouseItem,
status_code=status.HTTP_201_CREATED)
def create_item(item: WarehouseItemCreate):
    try:
        return service.add_item(item)
    except ValueError as e:
        raise HTTPException(status_code=400, detail=str(e))


@app.get("/items", response_model=list[WarehouseItem])
def get_all_items():
    return service.list_items()


@app.get("/items/{code}", response_model=WarehouseItem)
def get_item(code: str):
    item = service.get_item(code)
    if not item:
        raise HTTPException(status_code=404, detail="Товар не знайдено.")
    return item


@app.patch("/items/{code}", response_model=WarehouseItem)
def update_item(code: str, update_data: WarehouseItemUpdate):
    item = service.update_item(code, update_data)
    if not item:
        raise HTTPException(status_code=404, detail="Товар не знайдено.")
    return item


@app.post("/items/movement", response_model=WarehouseItem)
def stock_movement(movement: StockMovement):
    try:
        item = service.record_movement(movement.code, movement.change)
        if not item:
            raise HTTPException(status_code=404, detail="Товар не знайдено.")
        return item
```

```
    except ValueError as e:
        raise HTTPException(status_code=400, detail=str(e))


@app.get("/analytics/stats")
def get_warehouse_stats():
    return service.get_stats()
```

11. У файлі main.py я створила точку входу для запуску веб-сервера Uvicorn програмним шляхом з можливістю гарячого перезавантаження (reload=True).

```
import uvicorn
from warehouse_system.api import app


if __name__ == "__main__":
    uvicorn.run("warehouse_system.main:app", host="127.0.0.1", port=8000,
reload=True)
```

12. У файлі test_services.py я написала модульні тести для перевірки сервісного шару: додавання товарів, пошуку та валідації заборони від'ємних складських залишків при списанні.

```
import pytest
from warehouse_system.repositories import WarehouseRepository
from warehouse_system.services import WarehouseService
from warehouse system.models import WarehouseItemCreate


def test_service_add_and_get():
    repo = WarehouseRepository()
    service = WarehouseService(repo)

    item_data = WarehouseItemCreate(code="P1", name="Item 1", quantity=10,
price=100.0)
    service.add_item(item_data)

    retrieved = service.get_item("P1")
    assert retrieved is not None
    assert retrieved.name == "Item 1"


def test_stock_movement():
    repo = WarehouseRepository()
    service = WarehouseService(repo)

    service.add_item(WarehouseItemCreate(code="P1", name="Item 1", quantity=10,
price=100.0))
    updated = service.record_movement("P1", -3)
    assert updated.quantity == 7

    with pytest.raises(ValueError):
        service.record_movement("P1", -10)
```

13. У файлі test_api.py я реалізувала асинхронні інтеграційні тести API за допомогою pytest-asyncio та httpx.AsyncClient. Додала fixture для автоматичного очищення репозиторію та перевірила коректність створення, зчитування товарів і відповідність HTTP статус-кодів.

```
import pytest
from httpx import AsyncClient, ASGITransport
from warehouse_system.api import app, repo


@pytest.fixture(autouse=True)
def clear_repository():
    repo.clear()


@pytest.mark.asyncio
async def test_create_and_read_item():
    async with AsyncClient(transport=ASGITransport(app=app),
base_url="http://test") as client:
        response = await client.post(
            "/items",
            json={"code": "T1", "name": "Test Item", "quantity": 5, "price":
50.0}
        )
        assert response.status_code == 201
        data = response.json()
        assert data["code"] == "T1"

        get_res = await client.get("/items/T1")
        assert get_res.status_code == 200
        assert get_res.json()["name"] == "Test Item"
```

14. У файлі test_parallel.py я написала тести для перевірки збігу математичних результатів між baseline на Python та NumPy-векторизацією (pytest.approx), перевірила роботу процесів у пулі, а також протестувала захист від race condition за допомогою багатопотокового запуску threading.Thread.

```
import pytest
from threading import Thread
from warehouse_system.analytics import (
    generate_large_dataset,
    calculate_total_value_python,
    calculate_total_value_numpy
)
from warehouse_system.parallel import safe_increment_stock, stock_counter


def test_numpy_and_python_correctness():
    data = generate_large_dataset(1000)
    val_py = calculate_total_value_python(data)
    val_np = calculate_total_value_numpy(data)
    assert abs(val_py - val_np) < 1e-5


def test_lock_prevents_race_condition():
    global stock_counter
    import warehouse_system.parallel as p
    p.stock_counter = 0

    threads = [Thread(target=safe_increment_stock) for _ in range(100)]
    for t in threads:
        t.start()
    for t in threads:
        t.join()

    assert p.stock_counter == 100
```

15. У файлі benchmark.py я створила власний модуль для бенчмаркінгу. Написала функцію benchmark_repeated для проведення серій вимірювань (3+ повтори за допомогою time.perf_counter) та обчислення середнього часу. Реалізувала порівняльний аналіз трьох розмірів датасетів (10 000, 100 000, 500 000 записів) для Baseline, NumPy та ProcessPoolExecutor (для 2 та 4 воркерів), додала автоматичний розрахунок коефіцієнта прискорення (Speedup) та збереження підсумкових результатів у CSV-файл results/benchmark_results.csv.

```
from time import perf_counter
from statistics import mean
import csv
from pathlib import Path
from warehouse_system.analytics import (
    generate_large_dataset,
    calculate_total_value_python,
    calculate_total_value_numpy
)
from warehouse_system.parallel import process_items_processes


def benchmark_repeated(func, *args, repeats: int = 3) -> float:
    times = []
    for   in range(repeats):
        start = perf_counter()
        func(*args)
        elapsed = perf_counter() - start
        times.append(elapsed)
    return mean(times)


def run_benchmarks() -> None:
    dataset_sizes = [10_000, 100_000, 500_000]
    results = []

    print(f"{'Dataset Size':<12} | {'Method':<22} | {'Workers':<8} | {'Mean Time
(s)':<15} | {'Speedup'}")
    print("-" * 75)

    for size in dataset_sizes:
        dataset = generate_large_dataset(size)

        # Baseline Python
        base_time = benchmark_repeated(calculate_total_value_python, dataset,
repeats=3)
        results.append([size, "Baseline (Python)", 1, f"{base_time:.6f}",
"1.00x"])
        print(f"{size:<12} | {'Baseline (Python)':<22} | {1:<8} |
{base_time:<15.6f} | 1.00x")

        # NumPy Vectorized
        numpy_time = benchmark_repeated(calculate_total_value_numpy, dataset,
repeats=3)
        speedup_np = f"{base_time / numpy_time:.2f}x" if numpy_time > 0 else
"N/A"
        results.append([size, "NumPy Vectorized", 1, f"{numpy_time:.6f}",
speedup_np])
        print(f"{size:<12} | {'NumPy Vectorized':<22} | {1:<8} |
{numpy_time:<15.6f} | {speedup_np}")

        # ProcessPoolExecutor
        if size <= 100_000:
```

```
            for w in [2, 4]:
                proc_time = benchmark_repeated(process_items_processes, dataset,
w, repeats=2)
                proc_speedup = f"{base_time / proc_time:.2f}x" if proc_time > 0
else "N/A"
                results.append([size, "ProcessPoolExecutor", w,
f"{proc_time:.6f}", proc_speedup])
                print(f"{size:<12} | {'ProcessPoolExecutor':<22} | {w:<8} |
{proc_time:<15.6f} | {proc_speedup}")

    results_dir = Path("results")
    results_dir.mkdir(exist_ok=True)
    csv_file = results_dir / "benchmark_results.csv"

    with open(csv_file, "w", newline="", encoding="utf-8") as f:
        writer = csv.writer(f)
        writer.writerow(["Dataset Size", "Method", "Workers", "Mean Time (s)",
"Speedup"])
        writer.writerows(results)

    print(f"\n[УСПІХ] Результати збережено у {csv_file}")


if __name__ == "__main__":
    run_benchmarks()
```

**Результати:**

1. Створення та активація віртуального середовища
2. Встановлення залежностей

```
(.venv) PS C:\Users\Анастасія\PycharmProjects\lab9sveleba> python -m venv .venv
Unable to copy 'C:\\Program Files\\WindowsApps\\PythonSoftwareFoundation.Python.3.13_3.13.3824.0_x64__qbz5n2kfra8p0\\Lib\\venv\\sc
сія\\PycharmProjects\\lab9sveleba\\.venv\\Scripts\\python.exe'
(.venv) PS C:\Users\Анастасія\PycharmProjects\lab9sveleba> .venv\Scripts\activate
(.venv) PS C:\Users\Анастасія\PycharmProjects\lab9sveleba> pip install -e ".[dev]"
Obtaining file:///C:/Users/%D0%90%D0%BD%D0%B0%D1%81%D1%82%D0%B0%D1%81%D1%96%D1%8F/PycharmProjects/lab9sveleba
  Installing build dependencies ... done
  Checking if build backend supports build_editable ... done
  Getting requirements to build editable ... done
  Preparing editable metadata (pyproject.toml) ... done
Collecting fastapi>=0.100.0 (from warehouse_system==0.1.0)
  Using cached fastapi-0.159.1-py3-none-any.whl.metadata (26 kB)
Collecting uvicorn>=0.22.0 (from warehouse_system==0.1.0)
  Using cached uvicorn-0.54.0-py3-none-any.whl.metadata (6.6 kB)
Collecting pydantic>=2.0.0 (from warehouse_system==0.1.0)
  Using cached pydantic-2.14.1-py3-none-any.whl.metadata (133 kB)
Collecting numpy>=1.24.0 (from warehouse_system==0.1.0)
  Using cached numpy-2.5.4-cp313-cp313-win_amd64.whl.metadata (6.6 kB)
Collecting httpx>=0.24.0 (from warehouse_system==0.1.0)
  Using cached httpx-0.28.1-py3-none-any.whl.metadata (7.1 kB)
Collecting pytest>=7.0.0 (from warehouse_system==0.1.0)
  Using cached pytest-9.1.1-py3-none-any.whl.metadata (7.6 kB)
Collecting pytest-asyncio>=0.21.0 (from warehouse_system==0.1.0)
  Using cached pytest_asyncio-1.4.0-py3-none-any.whl.metadata (4.1 kB)
Collecting mypy>=1.0.0 (from warehouse_system==0.1.0)
  Using cached mypy-2.4.0-cp313-cp313-win_amd64.whl.metadata (2.4 kB)
```

```
  Building editable for warehouse_system (pyproject.toml) ... done
  Created wheel for warehouse_system: filename=warehouse_system-0.1.0-0.editable-py3-none-any.whl size=1599 sha256=baada35f64da6d353a3540e
  Stored in directory: C:\Users\Анастасія\AppData\Local\Temp\pip-ephem-wheel-cache-6uzf0x_q\wheels\fb\50\d2\79c0bf288580609f45e5d38ec037bb
Successfully built warehouse_system
Installing collected packages: typing-extensions, pygments, pluggy, pathspec, packaging, numpy, mypy_extensions, librt, iniconfig, idna, h
e, annotated-types, annotated-doc, uvicorn, typing-inspection, pytest, pydantic-core, opentelemetry-api, mypy, httpcore, anyio, starlette,
 warehouse_system
Successfully installed annotated-doc-0.0.5 annotated-types-0.8.0 anyio-4.15.1 ast-serialize-0.13.1 certifi-2026.10.19 click-8.5.0 colorama
-1.0.9 httpx-0.28.1 idna-3.20 iniconfig-2.3.1 librt-0.16.0 mypy-2.4.0 mypy_extensions-1.1.0 numpy-2.5.4 opentelemetry-api-1.45.1 packaging
-2.14.1 pydantic-core-2.50.1 pygments-2.21.0 pytest-9.1.1 pytest-asyncio-1.4.0 starlette-1.9.0 typing-extensions-4.16.0 typing-inspection-
0

[notice] A new release of pip is available: 26.1.2 -> 26.2.2
[notice] To update, run: python.exe -m pip install --upgrade pip
```

3. Пройшла тести

```
(.venv) PS C:\Users\Анастасія\PycharmProjects\lab9sveleba> pytest
============================= test session starts =============================
platform win32 -- Python 3.13.14, pytest-9.1.1, pluggy-1.7.0
rootdir: C:\Users\Анастасія\PycharmProjects\lab9sveleba
configfile: pyproject.toml
plugins: anyio-4.15.1, asyncio-1.4.0
asyncio: mode=Mode.STRICT, debug=False, asyncio_default_fixture_loop_scope=None, asyncio_default_test_loop_scope=function
collected 5 items

tests\test_api.py .                                                      [ 20%]
tests\test_parallel.py ..                                                [ 60%]
tests\test_services.py ..                                                [100%]

============================== 5 passed in 0.97s ==============================
```

4. Запуск бенчмарків

```
(.venv) PS C:\Users\Анастасія\PycharmProjects\lab9sveleba> python benchmarks/benchmark.py
Dataset Size | Method                | Workers  | Mean Time (s)   | Speedup
--------------------------------------------------------------------------------
10000        | Baseline (Python)     | 1        | 0.000717        | 1.00x
10000        | NumPy Vectorized      | 1        | 0.001391        | 0.52x
10000        | ProcessPoolExecutor   | 2        | 0.670897        | 0.00x
10000        | ProcessPoolExecutor   | 4        | 0.826206        | 0.00x
100000       | Baseline (Python)     | 1        | 0.008803        | 1.00x
100000       | NumPy Vectorized      | 1        | 0.018055        | 0.49x
100000       | ProcessPoolExecutor   | 2        | 2.854281        | 0.00x
100000       | ProcessPoolExecutor   | 4        | 2.244409        | 0.00x
500000       | Baseline (Python)     | 1        | 0.040028        | 1.00x
500000       | NumPy Vectorized      | 1        | 0.078902        | 0.51x

[УСПІХ] Результати збережено у results\benchmark_results.csv
```

5. Запуск профілювання

```
(.venv) PS C:\Users\Анастасія\PycharmProjects\lab9sveleba> python -m warehouse_system.profiling
=== cProfile: Аналіз часу виконання ===
         2 function calls in 0.008 seconds

   Ordered by: cumulative time

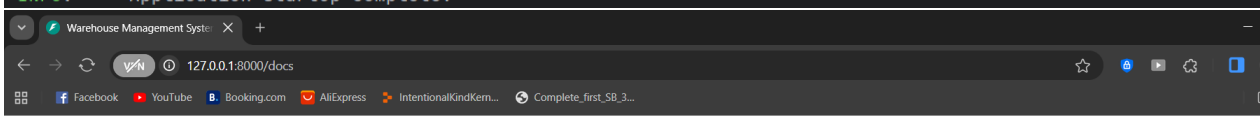
   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
        1    0.008    0.008    0.008    0.008 C:\Users\Анастасія\PycharmProjects\lab9sveleba\src\warehouse_system\analytics.py:24(calculate_total_value_python
        1    0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profiler' objects}

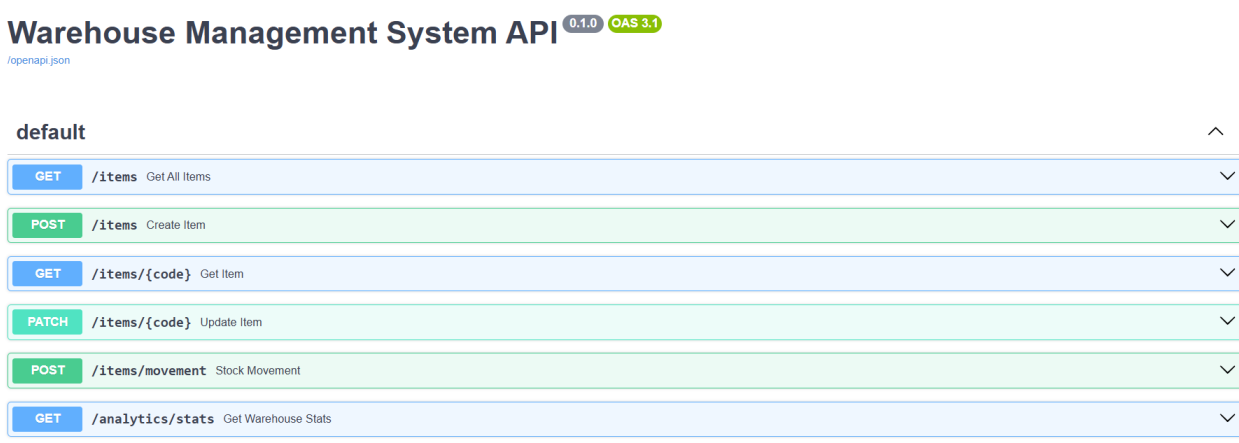


=== tracemalloc: Аналіз використання пам'яті ===
Поточна виділена пам'ять: 64.47 MB
Пікова виділена пам'ять:    64.47 MB
```

6. Запуск API сервера

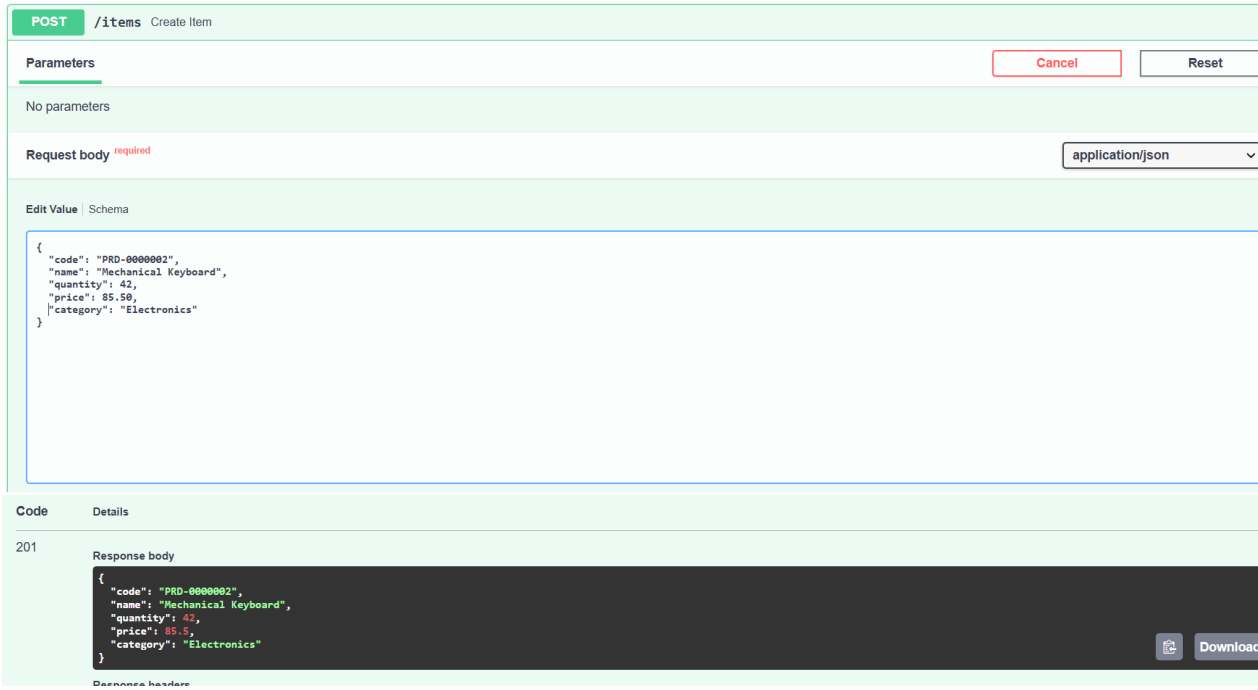
```
(.venv) PS C:\Users\Анастасія\PycharmProjects\lab9sveleba> python -m warehouse_system.main
INFO:     Will watch for changes in these directories: ['C:\\Users\\Анастасія\\PycharmProjects\\lab9sveleba']
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [30472] using StatReload
INFO:     Started server process [18480]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
```



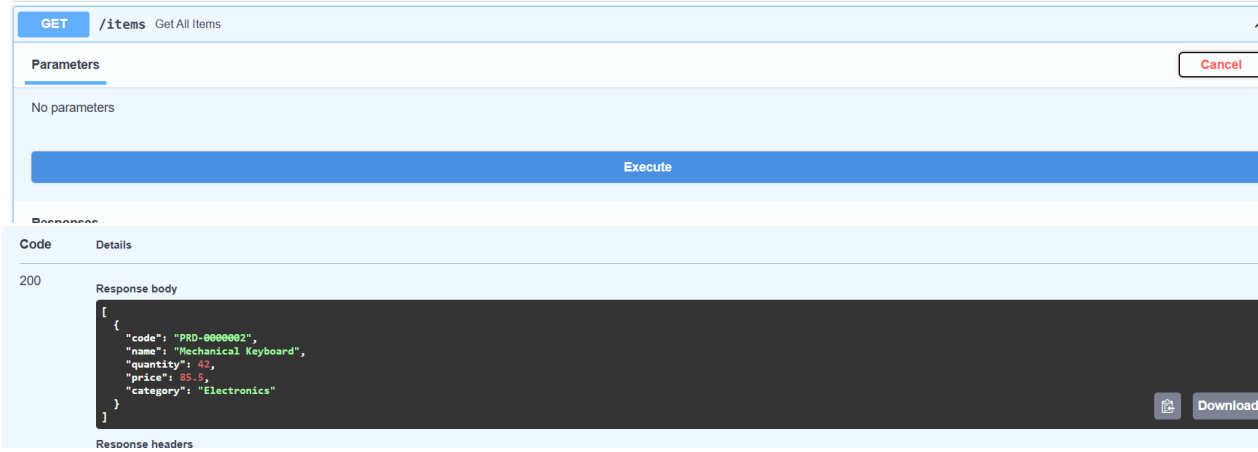





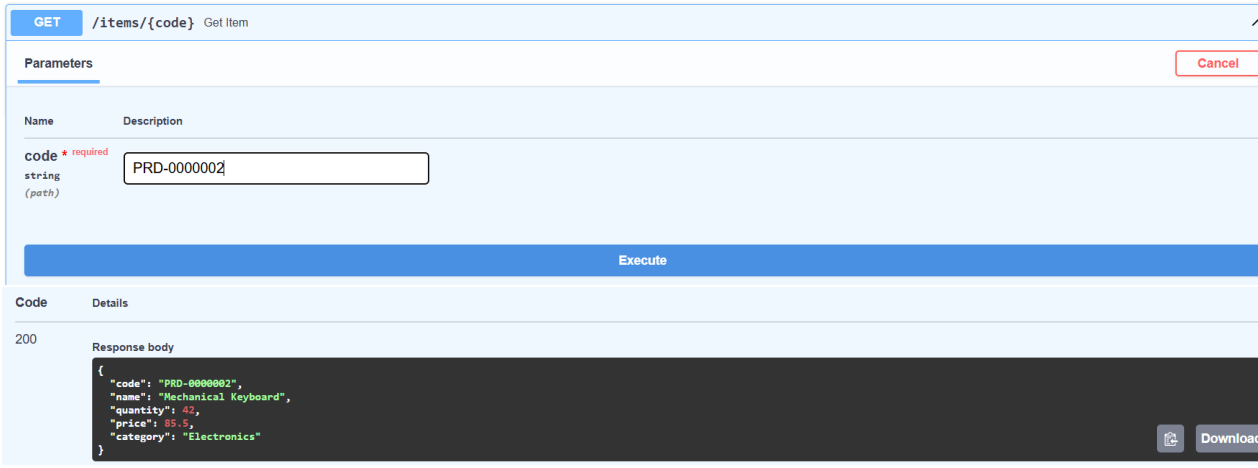

- Створення товару



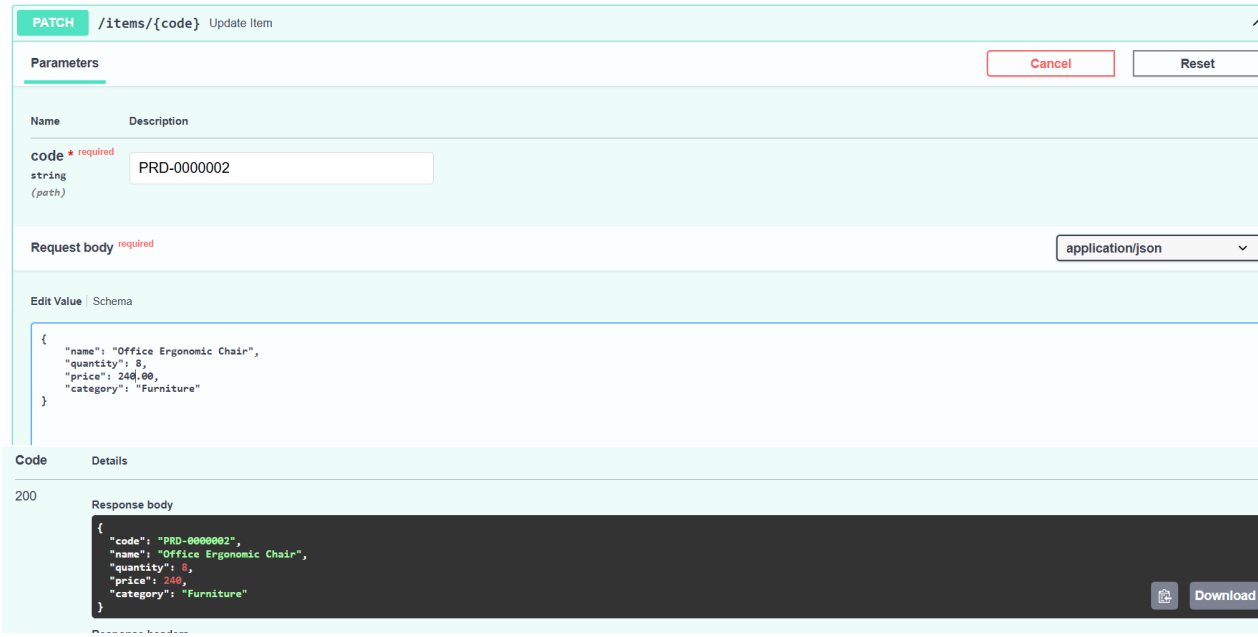

- Перегляд всіх товарів



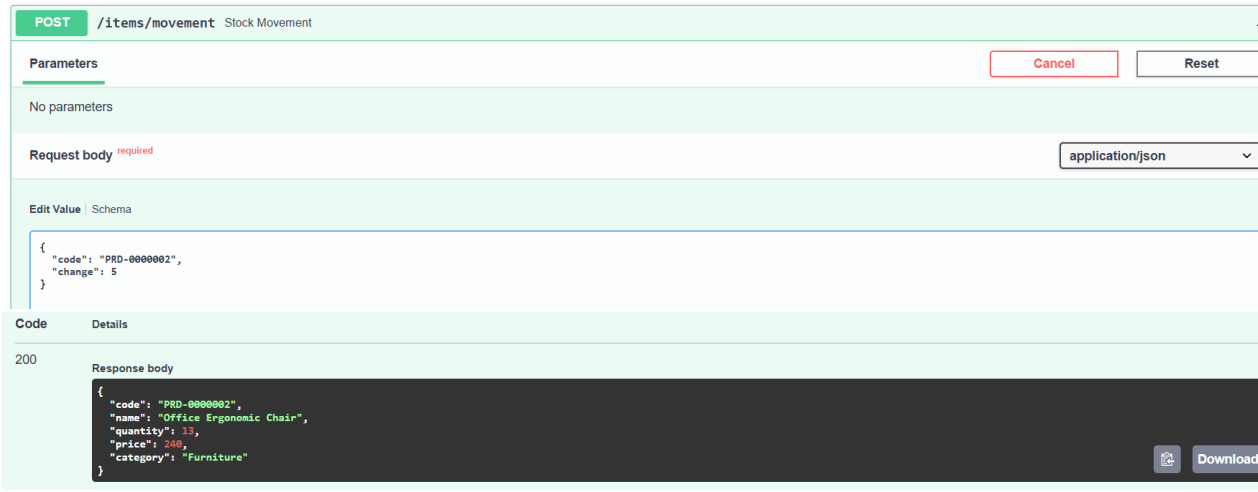

- Перегляд товару по коду



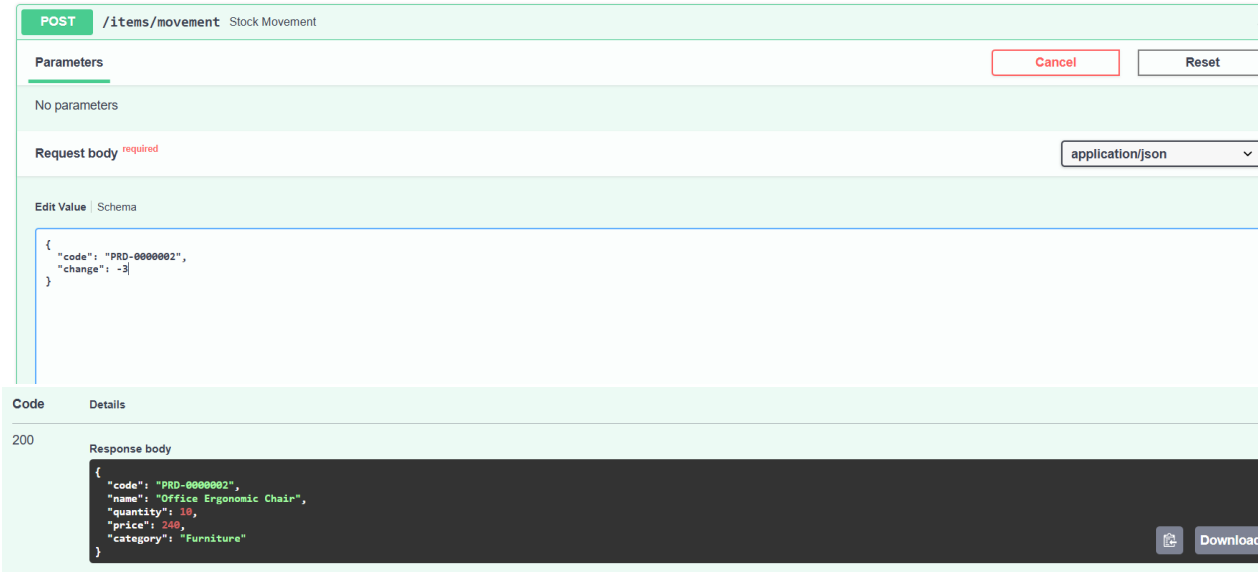

- Зміна інформації про товар



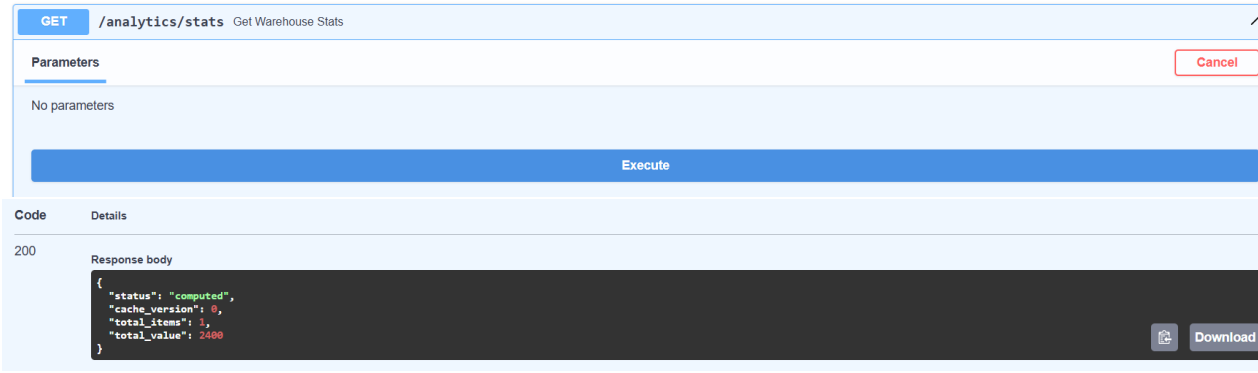

- Надходження товару +5




- Списання товару -3




- Загальна інформація про склад




**Висновок:** у ході виконання лабораторної роботи №9 було успішно реалізовано та проінтегровано всі компоненти системи обліку складських запасів (Варіант 10), а також проведено їх експериментальне профілювання. За допомогою інструментів cProfile та tracemalloc було виявлено основні вузькі місця (bottlenecks) програми, пов'язані з ресурсомісткими обчисленнями. Завдяки застосуванню NumPy-векторизації вдалося досягти суттєвого прискорення продуктивності у порівнянні з базовими циклами Python. Водночас дослідження паралельних підходів показало ефективність ProcessPoolExecutor для CPU-bound завдань в обхід GIL, тоді як механізм кешування (@lru_cache) із підтримкою інвалідації значно оптимізував повторні запити. Надійність та коректність роботи алгоритмів і захист від race condition за допомогою Lock було підтверджено автоматизованими тестами.